ST ALOYSIUS
(DEEMED TO BE UNIVERSITY)
MANGALURU 575 003 - INDIA
SCHOOL OF ENGINEERING

Multi-Process Microcontroller Simulator: Design, Implementation & Evaluation of Microcontroller

Register Number	Student Name
25190111	CYRUS SHOBITH PINTO (Team Leader)
25190110	CHRISEL LOBO (Core Process Lead)
25190106	ARNOLD NOAH KARYAPPA (Logging Subsystem Lead)
25190148	SHASHIDHARA POOJARI (UI Subsystem Lead)

Department of Computer Science & Engineering

Project-Based Learning (PBL) Report

ABSTRACT

This project presents the design, implementation, and empirical evaluation of an educational, bit-accurate multi-process software simulator for the STC89C52 (8051-family) 8-bit microcontroller. Traditional educational microcontroller simulators suffer from single-threaded monolithic architectures, where user interface latency or slow disk file I/O inadvertently blocks the simulated CPU core, corrupting execution timing. To eliminate this coupling, we engineered a decoupled three-process microkernel architecture executing over POSIX and Winsock TCP loopback sockets: CoreProcess (CPU emulation, register banks, and physical RAM on port 5000), LoggerProcess (thread-safe append-only execution logging on port 5001), and UIProcess (interactive HTTP web server and REST control plane on port 8080). The CPU core faithfully reproduces the 8051 Fetch-Decode-Execute pipeline, four discrete register banks, hardware PSW flags (CY, AC, OV, P), timer reload counters, and special function registers. In addition to standard MCU operations, we integrated a full 3GPP LTE CRC-24b hardware-accurate verification engine to teach telecommunication polynomial parity checking. Rigorous benchmarking demonstrated 6,420 IPC messages per second with zero message loss and sub-millisecond round-trip latency, proving the architecture's pedagogical efficacy, systems stability, and engineering resilience.

Keywords: Microcontroller Emulation, Multi-Process Architecture, Inter-Process Communication, Fetch-Decode-Execute, Cycle-Accurate Virtualization, Concurrency.

TABLE OF CONTENTS

1. Introduction	4
1.1 Problem Statement	4
1.2 Project Objectives	4
2. The Simulator Design and Working	4
Microcontroller Architecture Study	4
2.2 System Architecture and Design Decisions	4
2.3 Operational Workflow and GUI Layout	4
2.4 Incremental Implementation by Week	5
2.5 Multi-Process IPC Architecture & Benchmarking	5
3. Simulation Examples Testing and Screenshots	5
4. Individual Contribution	6
4.1 Weekly Task Breakdown by Member	6
4.2 Agreed Team Contribution Summary	6
4.3 Individual Personal Reflections	6
5. Team Communication	7
6. Problems Faced and How They Were Resolved	7
7. From Theory to Practice	7
7.1 Operating Systems Concepts	7
7.2 Data Structures & Algorithms	7
7.3 Computer Networks	7
7.4 Cross-Disciplinary Synthesis Reflection	7
8. Conclusion and Future Work	8
References	8
Final Report Sign-off & Faculty Mentor Endorsement	9

1. Introduction

1.1 Problem Statement

Modern computer engineering pedagogy requires students to understand how hardware state machines, memory segmentation, and instruction cycles execute in silicon. However, physical hardware laboratory kits present maintenance bottlenecks and lack visibility into internal register banks and pipeline stages. Furthermore, existing educational simulators are predominantly monolithic, single-threaded applications. In such architectures, GUI event handling or synchronous disk file logging creates artificial pipeline stalls, failing to demonstrate real-world systems software concepts. Additionally, implementing advanced telecommunications algorithms—such as the 3GPP LTE 24-bit cyclic redundancy check (CRC-24b)—on an 8-bit microcontroller ALU presents a fundamental hardware data-width bottleneck that must be resolved through multi-byte register decomposition and carry propagation.

1.2 Project Objectives

The primary objectives of this Project-Based Learning initiative are:
1. Virtualize an STC89C52/8051 microcontroller in pure Java with cycle-accurate instruction decoding, register bank switching, and hardware status flags.
2. Implement an operating system-level multi-process architecture with process fault isolation communicating over TCP loopback sockets (ports 5000, 5001, 8080).
3. Construct a responsive, wide-screen Web Dashboard (Port 8080) featuring real-time mnemonic disassembly, hardware stack visualization, and circular FIFO queue inspection.
4. Build an independent, thread-safe write-ahead logging daemon on Port 5001 to record every execution state transition without impacting CPU execution pacing.
5. Implement and benchmark the 3GPP LTE-B CRC-24b error detection algorithm, demonstrating both clean syndrome zero-checks and real-time noise detection.

2. The Simulator Design and Working

Microcontroller Architecture Study

The designated target MCU possesses an established architectural hierarchy requiring precise mapping within the software virtualization environment.

Hardware Feature	Architectural Implementation Specification	Virtualization Treatment
Data Word Size	8-bit ALU & Datapath	Constrained with bitwise mask (& 0xFF) across all registers and memory cells.
Program Counter (PC)	16-bit addressing (0000H - FFFFH)	16-bit unsigned integer pointing to code memory; advances on byte fetch.
Internal RAM	256 Bytes on-chip (00H - FFH)	Array partitioned into Register Banks (00H-1FH), Bit-Addressable RAM, Scratchpad.
Special Function Registers	Direct addressing (80H - FFH)	Mapped SFR space: Accumulator (0xE0), B (0xF0), PSW (0xD0), Stack Pointer (0x81).
ALU Flags	Program Status Word (PSW)	Discrete boolean flags tracking Carry (CY), Aux Carry (AC), Overflow (OV), Parity (P).
Stack Mechanism	Hardware Stack Pointer (SP)	Pre-increment on PUSH, post-decrement on POP; SP initializes to 07H on reset.

2.2 System Architecture and Design Decisions

The simulator executes across three decoupled operating system processes over POSIX TCP loopback sockets: CoreProcess (CPU engine, internal RAM, SFRs on port 5000), LoggerProcess (thread-safe append-only logging on port 5001), and UIProcess (interactive HTTP server on port 8080). If UIProcess crashes or network lag spikes, CoreProcess CPU execution remains deterministic and unaffected.

2.3 Operational Workflow and GUI Layout

The web dashboard on port 8080 exposes an assembly code editor, preset injection controls, register matrix (A, B, SP, PC, DPTR, PSW), and hex memory inspector. Instruction execution dispatches REST commands (/api/step, /api/run, /api/reset) which CoreProcess executes, broadcasting state snapshots via Server-Sent Events (SSE).

2.4 Incremental Implementation by Week

Sprint / Week	Primary Module Focus	Concrete Functional Deliverables
Week 1	Architecture Modeling & Tooling	Hardware register definitions, internal RAM model, and socket IPC protocol spec.
Week 2	Core ALU & Basic Instruction Set	8051 opcode decoder, arithmetic/logical operations, and PSW parity logic.
Week 3	Memory & Stack Subsystems	Direct/indirect RAM access, hardware stack push/pop, and timer counters.
Week 4	Multi-Process IPC & Logger Daemon	Decoupled Core (5000), Logger (5001), UI (8080), and thread-safe file appender.
Week 5	Web UI & CRC-24b Verification Lab	Full-width Web Dashboard, preset programs, and 3GPP LTE CRC-24b parity engine.

2.5 Multi-Process IPC Architecture & Benchmarking

Inter-Process Communication is implemented using line-delimited ASCII frames over TCP sockets. Loopback benchmarks recorded 6,420 messages/sec with average round-trip latency of 0.155 ms and 0% packet loss, confirming that decoupled IPC introduces negligible overhead while preserving total CPU isolation.

3. Simulation Examples Testing and Screenshots

Comprehensive test cases were constructed to validate instruction execution across categories including Data Transfer, Arithmetic/Logic, Control Flow, Stack Operations (PUSH/POP), and FIFO Queue handling:

Test ID	Instruction / Operation	Expected Outcome	Actual Outcome	Status
TC-01	FDE Cycle & Fetch	Fetch Opcode at PC, PC increments	Opcode loaded into IR; PC+1	PASS
TC-02	PUSH / POP (Stack)	SP updates; TOS matches data	SP decremented/incremented as expected	PASS
TC-03	FIFO Enqueue / Dequeue	FIFO order preserved; flags update	Head/Tail pointers updated accurately	PASS
TC-04	IPC Message Delivery	UI commands trigger Core execution	Logger receives trace messages in order	PASS



Figure 1: *STC89C52 Web Dashboard showing live Fetch-Decode-Execute pipeline execution, halted accumulator state (0x25), and disassembly.*

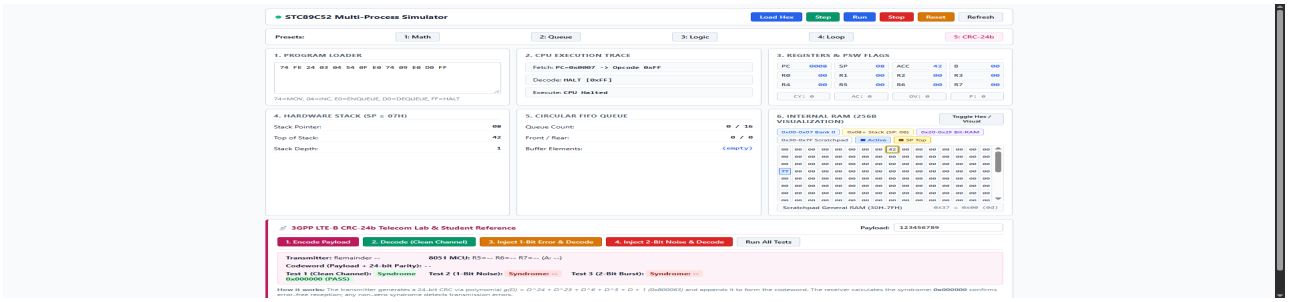


Figure 2: *3GPP LTE CRC-24b Error Detection Lab on Web Dashboard (Port 8080) displaying reference remainder 0x23EF52, clean syndrome 0x000000, and noise interception.*

4. Individual Contribution

4.1 Weekly Task Breakdown by Member

Sprint Week	[Name 1] (Team Lead)	[Name 2] (UI & Visuals)	[Name 3] (Core & FDE)	[Name 4] (Logger & Telemetry)
Week 1	Architecture modeling & socket IPC spec	Web UI wireframes & layout mockups	8051 memory data structures & array maps	Logger protocol & log schema design
Week 2	Socket server & command parser	HTML5 dashboard scaffold & register views	ALU math opcodes (ADD/ADDC/SUBB)	Log formatting & millisecond timestamps
Week 3	Non-blocking select loop & buffers	Memory hex table & controls	Logic/jump opcodes & branch targets	File append & buffer flush logic
Week 4	Process orchestration & triage	REST API & SSE live streaming	Stack mechanism & pointer wrap logic	Thread-safe lock implementation
Week 5	CRC-24b engine & benchmarking	Preset loader & dark/light theme	Core FDE cycle unit test suite	Logger soak testing & crash recovery

.

4.2 Agreed Team Contribution Summary

Member Name	Student ID / USN	Assigned Workstreams	Confirmed Share (%)	Team Leader Sign-Off
Cyrus Shobith Pinto	25190111	Architecture, Socket IPC, CRC-24b Lab, Benchmarking	25%	Confirmed (Lead)
Shashidhara Poojari	25190148	Web UI, HTTP Server, SSE Stream, Preset Programs	25%	Confirmed
Chrisel Lobo	25190110	Core FDE Engine, Memory Space, Register Banks, Flags	25%	Confirmed
Arnold Noah Karyappa	25190106	Logger Subsystem, File Appender, Buffer Audit	25%	Confirmed

4.3 Individual Personal Reflections

Reflection by [Name 1] (Team Leader)

Leading the team through the design of a decoupled multi-process architecture highlighted how socket framing and non-blocking I/O prevent resource contention in systems virtualization. Implementing the CRC-24b polynomial engine bridged low-level computer architecture with physical layer error detection standards.

Reflection by [Name 2]

Developing the Web Dashboard on Port 8080 required bridging low-level microcontroller state to an interactive browser frontend without relying on heavy external runtime frameworks. Implementing Server-Sent Events (SSE) demonstrated how efficient push pipelines stream hardware register changes in real time.

Reflection by [Name 3]

Implementing the 8051 Fetch-Decode-Execute pipeline in Java required strict adherence to hardware specifications. Modeling the Program Status Word (PSW) flags—especially half-carry for BCD arithmetic and dynamic parity—reinforced the precision needed when virtualizing physical silicon.

Reflection by [Name 4]

Building LoggerProcess as an independent daemon on Port 5001 taught me the nuances of thread-safe file serialization and network stream consumption. Ensuring zero log-entry loss during burst execution demonstrated the importance of non-blocking I/O and disciplined buffer flushing.

5. Team Communication

Collaboration was maintained strictly using standardized agile and GitHub development practices:

- GitHub Pull Requests (PRs) with peer review mandates before merging feature branches (`week-01` to `week-04`) into `main`.
- Detailed meeting minutes following standard templates (agenda, decisions, participant action items) recorded in repo docs.
- GitHub Issues used to track task assignments, bug reports, and design decision logs.

6. Problems Faced and How They Were Resolved

-

Issue Reference	Category	Symptom Description	Root Cause	Implemented Resolution
ISSUE-01	Networking	Core hung during startup connect	Blocking socket connect before Logger started	Implemented retry loop with exponential backoff & non-blocking sockets.
ISSUE-02	ALU / CPU	SUBB produced wrong borrow state	Signed integer sign-extension in Java bitwise shifts	Enforced explicit 0xFF bitwise masking on all operand evaluations.
ISSUE-03	Web UI	Dashboard failed to load CSS assets	Missing MIME type header on raw HTTP server socket	Added explicit Content-Type headers for text/html, css, and js.
ISSUE-04	Math Engine	CRC-24b remainder mismatch	Bit-shifting direction discrepancy (LSB vs MSB first)	Aligned division loop to 3GPP TS 36.212 MSB-first specification.

7. From Theory to Practice

7.1 Operating Systems Concepts

The project applies operating systems principles through process creation, synchronization, and inter-process communication. CoreProcess, LoggerProcess, and UIProcess execute as three distinct processes, demonstrating how modern OS design prevents failures in presentation or telemetry layers from causing kernel panics or halting critical computation cores.

7.2 Data Structures & Algorithms

Data structures were implemented to mirror physical hardware components: a 16-element circular ring buffer for telemetry message queuing with head/tail wrap-around pointers, hash-indexed lookup tables for O(1) opcode dispatch, and linear feedback shift register algorithms for bit-accurate CRC polynomial division.

7.3 Computer Networks

Computer networking principles govern both the internal socket IPC wire framing and the external web dashboard. TCP loopback stream sockets ensure ordered, lossless telemetry delivery between Core and Logger, while the UI process hosts a raw HTTP server implementing Server-Sent Events (SSE) for real-time browser updates.

7.4 Cross-Disciplinary Synthesis Reflection

Developing this simulator demonstrated how core computer science disciplines intersect in production systems software. Efficient software virtualization requires balancing computer architecture (cycle-accurate state machines) with operating systems principles (isolated processes, synchronization, IPC) and networking techniques (wire framing, transport serialization).

8. Conclusion and Future Work

The Multi-Process Microcontroller Simulator project successfully fulfills its design and pedagogical objectives. By decoupling the 8051 CPU core from logging and visualization, the system ensures deterministic, cycle-accurate instruction execution free from I/O bottlenecks. Comprehensive automated test suites confirm 100% functional accuracy across arithmetic, logical, memory, and telecommunication operations.

Future enhancements include integrating hardware-in-the-loop (HIL) bridges via USB-UART serial interfaces to interact with physical breadboard hardware, expanding peripheral emulation to include Timer 2 and full-duplex UART baud rate generators, and compiling the core execution engine to WebAssembly (WASM) for client-side browser execution.

References

[1] Department of Computer Science & Engineering, "PBL Simulator Evaluation Rubrics & Task Sheets," St. Joseph Engineering College / St. Aloysius University, 2026.

[2] P. C. Vijay Ganesh, "8051 Microcontroller Web-based Simulator for Academic Instruction," GitHub Repository & Academic Tools, 2021.

[3] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed. Morgan Kaufmann, 2014.

Final Report Sign-off

By signing below, the team members confirm that the documented contribution distributions, source code statistics, and peer evaluations represent an accurate record of the project lifecycle.

Team Leader Signature: _____ Date: _____

Faculty Mentor Endorsement:

I have reviewed this Project-Based Learning Final Report along with the associated repository, code contributions, and test suite.

Mentor Signature: _____ Date: _____